Build a Custom MCP Client with Python

Estimated time needed: 45 minutes

In this lab, you'll build a minimal Python-based Model Context Protocol (MCP) client. You'll connect to an MCP server via STDIO transport, interact with its tools, resources, and prompts through a simple command-line interface.

By the end of this lab, you'll have a lightweight MCP client that demonstrates the core concepts you can extend for your own applications.

Learning Objectives

After completing this lab, you will be able to:

- Connect to an MCP server using STDIO transport
- Discover and invoke MCP tools
- Read MCP resources via URIs
- Execute MCP prompt templates
- Handle MCP protocol sessions properly

Prerequisites

Before starting this lab, you should have:

- Basic Python programming knowledge
- Understanding of MCP architecture (client-server model)
- Familiarity with async/await patterns in Python

The content of this lab is licensed under Apache 2.0

- Awareness of MCP concepts: tools, resources, and prompts

The content of this lab is licensed under Apache 2.0

Lab Setup

Let's set up your development environment.

Create Virtual Environment

bash

```bash
python3.11 -m venv mcp_client_env
source mcp_client_env/bin/activate
```

Run

Install Dependencies

Install the MCP SDK and FastMCP for simplified server creation:

bash

```bash
pip install mcp==1.16.0 fastmcp==2.12.5
```

Run

Create Project Structure

bash

```bash
mkdir mcp_client_lab
cd mcp_client_lab
mkdir resources
```

Run

The content of this lab is licensed under Apache 2.0

We'll create the server, client, and resource files in the next steps.

The content of this lab is licensed under Apache 2.0

Create Resource Files

Let's create some real resource files that the MCP server will expose.

Create project_info.txt

Click below to open the file editor:

Open project_info.txt in IDE

Add the following content:

plaintext

```
Project Name: MCP Client Lab
Version: 1.0.0
Description: A hands-on lab for building MCP clients with Python
Author: Skills Network
Status: Active
```

Save the file (`File` → `Save` or `Ctrl+S`).

Create README.md

Click below to open the file editor:

Open README.md in IDE

Add the following content:

The content of this lab is licensed under Apache 2.0

markdown

```markdown
# MCP Client Lab

This lab teaches you how to build Model Context Protocol (MCP) clients

## What You'll Learn
- STDIO transport connections
- Tool invocation patterns
- Resource reading via URIs
- Prompt template usage

## Getting Started
Follow the lab instructions to build your first MCP client!
```

Save the file (`File` → `Save` or `Ctrl+S`).

These files will be exposed as MCP resources that your client can read.

The content of this lab is licensed under Apache 2.0

Build the MCP Server - Setup

Now let's build a simple MCP server using FastMCP.

Click below to open the server file:

Open mcp_server.py in IDE

Step 1: Add imports and logging configuration

python

```python
import logging
from fastmcp import FastMCP
from pathlib import Path

# Suppress verbose FastMCP logging
logging.getLogger("fastmcp").setLevel(logging.WARNING)

mcp = FastMCP("lab-server")
BASE_DIR = Path.cwd()
```

What this does: Imports FastMCP for simplified server creation, suppresses verbose INFO logs to keep output clean, and initializes the server with a name and base directory.

Step 2: Add the echo tool

python

```python
@mcp.tool()
def echo(text: str) -> str:
    """Echo back the input text."""
    return f"Echo: {text}"
```

The content of this lab is licensed under Apache 2.0

What this does: Creates a simple tool that echoes back any text input. The `@mcp.tool()` decorator automatically generates the JSON schema from the Python type hints.

Step 3: Add the write_file tool

python

```python
@mcp.tool()
def write_file(path: str, content: str) -> str:
    """Write content to a file."""
    file_path = BASE_DIR / path
    file_path.parent.mkdir(parents=True, exist_ok=True)
    file_path.write_text(content, encoding="utf-8")
    return f"Successfully wrote to {path}"
```

What this does: Creates a tool that writes content to a file. It handles creating parent directories if needed and returns a success message.

Step 4: Add the resource template

python

```python
@mcp.resource("file://resources/{filename}")
def read_resource_file(filename: str) -> str:
    """Read a file from the resources directory."""
    file_path = BASE_DIR / "resources" / filename
    if not file_path.exists():
        return f"File not found: {filename}"
    return file_path.read_text(encoding="utf-8")
```

What this does: Creates a resource template that exposes files from the resources directory. The URI pattern `file://resources/{filename}` allows clients to read any file by substituting the filename parameter.

The content of this lab is licensed under Apache 2.0

Step 5: Add the prompt template

```python
@mcp.prompt()
def review_file(filename: str) -> str:
    """Generate a prompt to review a file's contents."""
    return f"""Please review the file '{filename}' and provide:

1. A summary of its contents
2. Key points or sections
3. Any suggestions for improvement
4. Overall quality assessment

Use the appropriate tools to read the file if needed."""
```

What this does: Creates a prompt template that generates structured instructions for reviewing a file. This could be sent to an LLM to perform the review.

Step 6: Add the entry point

```python
if __name__ == "__main__":
    mcp.run()
```

What this does: Runs the MCP server when the script is executed. FastMCP automatically handles the STDIO transport setup.

☐ Click below to see the complete code for `mcp_server.py`

▶ Full code

Save the file. Your MCP server is complete!

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Understanding FastMCP

FastMCP simplifies MCP server creation significantly:

Traditional MCP Server:

- Define tool schemas manually with JSON
- Implement `list_tools()`, `call_tool()` handlers
- Handle resource URIs and content types
- Set up STDIO transport manually

FastMCP Server:

- Use `@mcp.tool()` decorator - schema generated automatically
- Use `@mcp.resource()` decorator with URI templates
- Use `@mcp.prompt()` decorator for templates
- Call `mcp.run()` - transport handled automatically

FastMCP uses Python type hints to generate tool schemas, making your code cleaner and reducing boilerplate.

The content of this lab is licensed under Apache 2.0

Understanding STDIO Transport

Before building the client, let's discuss the STDIO transport briefly.

STDIO (Standard Input/Output) lets MCP clients and servers communicate through standard streams:

- Client launches server as a subprocess
- Client sends JSON-RPC messages via server's **stdin**
- Server responds via **stdout**
- Communication is local, low-latency, and simple to debug

Why STDIO?

- Perfect for local integrations
- No network configuration needed
- Easy to launch and manage
- Ideal for development and testing

Now let's build the client!

The content of this lab is licensed under Apache 2.0

Build the MCP Client - Setup

Click below to open the client file:

Open mcp_client.py in IDE

Step 1: Add imports and class setup

python

```python
import asyncio
import sys
import json
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


class MCPClient:
    def __init__(self):
        self.session = None
        self.exit_stack = AsyncExitStack()
```

What this does: Imports the necessary MCP SDK components and creates a client class with session and cleanup management.

The content of this lab is licensed under Apache 2.0

Build the MCP Client - Connection

Step 2: Add the connection method

Add this method to the `MCPClient` class in `mcp_client.py`:

python

```python
    async def connect(self, server_script: str):
        server_params = StdioServerParameters(
            command="python",
            args=[server_script],
            env=None
        )

        stdio_transport = await self.exit_stack.enter_async_context(
            stdio_client(server_params)
        )
        read, write = stdio_transport

        self.session = await self.exit_stack.enter_async_context(
            ClientSession(read, write)
        )

        await self.session.initialize()
        print("✓ Connected to MCP server")
```

What this does:

1. Configures how to launch the server subprocess (Python + script path)
2. Creates STDIO transport and gets read/write streams
3. Creates MCP session with those streams
4. Performs initialization handshake with the server

The `exit_stack` ensures everything closes properly when done.

The content of this lab is licensed under Apache 2.0

Build the MCP Client - Core Operations

This is the core of your MCP client - these methods enable communication with the MCP server.

Step 3a: Add tool discovery and invocation methods

Add these methods to the `MCPClient` class in `mcp_client.py`:

python

```python
    async def list_tools(self):
        result = await self.session.list_tools()
        return result.tools
```

What this does: Queries the server to discover all available tools. The server responds with a list of tool definitions including names, descriptions, and input schemas.

Now add the tool invocation method:

python

```python
    async def call_tool(self, tool_name: str, arguments: dict):
        result = await self.session.call_tool(tool_name, arguments)
        return result
```

What this does: Executes a specific tool on the server with provided arguments. The server runs the tool logic and returns the result. This is how your client performs actions such as writing files or echoing text.

Step 3b: Add resource discovery and reading methods

Add these methods to the `MCPClient` class in `mcp_client.py`:

The content of this lab is licensed under Apache 2.0

python

```python
    async def list_resources(self):
        result = await self.session.list_resource_templates()
        return result.resourceTemplates
```

What this does: Retrieves resource templates from the server. Templates define URI patterns (such as `file://resources/{filename}`) that clients can use to access multiple resources. This is different from listing individual resources - you're getting the patterns, not the actual data.

Now add the resource reading method:

python

```python
    async def read_resource(self, uri: str):
        result = await self.session.read_resource(uri)
        return result
```

What this does: Fetches the actual content of a resource using its URI. The client provides a concrete URI (such as `file://resources/README.md`), the server matches it to a template pattern, extracts parameters, and returns the resource content.

Step 3c: Add prompt discovery and retrieval methods

Add these methods to the `MCPClient` class in `mcp_client.py`:

python

```python
    async def list_prompts(self):
        result = await self.session.list_prompts()
        return result.prompts
```

What this does: Discovers available prompt templates on the server. Prompts are reusable message templates that can guide LLM interactions. The server returns prompt names, descriptions, and required arguments.

The content of this lab is licensed under Apache 2.0

python

```python
    async def get_prompt(self, prompt_name: str, arguments: dict):
        result = await self.session.get_prompt(prompt_name, arguments)
        return result
```

What this does: Retrieves a rendered prompt template with your provided arguments. The server fills in the template parameters and returns structured messages that can be sent to an LLM. This enables consistent, parameterized prompting patterns.

Why these six methods matter: Together, these methods give your client complete access to all three MCP primitives - tools (actions), resources (data), and prompts (LLM guidance). Every MCP client needs these capabilities to be useful.

The content of this lab is licensed under Apache 2.0

Build the MCP Client - CLI Interface

Step 4: Add the interactive CLI

Add this method to the `MCPClient` class in `mcp_client.py`:

The content of this lab is licensed under Apache 2.0

python

```python
    async def run(self):
        print("\n=== MCP Client ===")
        print("Commands: tools | call | resources | read | prompts | p

        while True:
            cmd = input("> ").strip().lower()

            if cmd == "quit":
                break

            try:
                if cmd == "tools":
                    tools = await self.list_tools()
                    for t in tools:
                        print(f"  • {t.name}: {t.description}")

                elif cmd == "call":
                    tool_name = input("  Tool name: ").strip()
                    print("  Arguments (as JSON, for example, {\"text\
                    args = json.loads(input("  ").strip())
                    result = await self.call_tool(tool_name, args)
                    for content in result.content:
                        if hasattr(content, 'text'):
                            print(f"  Result: {content.text}")

                elif cmd == "resources":
                    resources = await self.list_resources()
                    if resources:
                        for r in resources:
                            name = getattr(r, 'name', getattr(r, 'desc
                            uri_template = getattr(r, 'uriTemplate', g
                            print(f"  • {name}")
                            print(f"    URI template: {uri_template}")
                    else:
                        print("  No resources available")

                elif cmd == "read":
                    uri = input("  URI: ").strip()
                    result = await self.read_resource(uri)
                    for content in result.contents:
                        if hasattr(content, 'text'):
                            print(f"\n{content.text}")

                elif cmd == "prompts":
                    prompts = await self.list_prompts()
```

The content of this lab is licensed under Apache 2.0

```
                    args_info = ""
                    if p.arguments:
                        arg_names = [arg.name for arg in p.argumen
                        args_info = f" (args: {', '.join(arg_names
                    print(f"  • {p.name}: {p.description}{args_inf

                elif cmd == "prompt":
                    prompt_name = input("  Prompt name: ").strip()
                    print("  Arguments (as JSON): ")
                    args = json.loads(input("  ").strip())
                    result = await self.get_prompt(prompt_name, args)
                    print(f"\n--- Prompt: {result.description} ---")
                    for msg in result.messages:
                        content_text = msg.content.text if hasattr(msg
                        print(f"{msg.role}: {content_text}")

                else:
                    print("  Unknown command")

            except json.JSONDecodeError:
                print("  Error: Invalid JSON format")
            except Exception as e:
                print(f"  Error: {e}")
```

What this does: Creates a simple command-line interface that:

- Shows available commands
- Maps each command to an MCP operation
- Handles user input and displays results
- Catches JSON parsing errors gracefully

The content of this lab is licensed under Apache 2.0

Build the MCP Client - Cleanup and Main

Step 5: Add cleanup and entry point

Add these methods to the `MCPClient` class and the main entry point in `mcp_client.py`:

python

```python
    async def cleanup(self):
        await self.exit_stack.aclose()


async def main():
    if len(sys.argv) < 2:
        print("Usage: python mcp_client.py <server_script>")
        sys.exit(1)

    client = MCPClient()
    try:
        await client.connect(sys.argv[1])
        await client.run()
    except KeyboardInterrupt:
        print("\nExiting...")
    finally:
        await client.cleanup()


if __name__ == "__main__":
    asyncio.run(main())
```

What this does:

- `cleanup()`: Closes all connections and resources
- `main()`: Establishes the entry point that connects to the server, runs the CLI, and ensures cleanup

The content of this lab is licensed under Apache 2.0

☐ Click below to see the complete code for `mcp_client.py`

▶ Full code

Save the file. Your minimal MCP client is complete!

The content of this lab is licensed under Apache 2.0

Test Your Client - Getting Started

Now let's test the client with the server.

Run the Client

In the terminal, run:

```bash
python mcp_client.py mcp_server.py
```

Run

You should see this response:

```plaintext
✓ Connected to MCP server

=== MCP Client ===
Commands: tools | call | resources | read | prompts | prompt | quit

>
```

The content of this lab is licensed under Apache 2.0

Test Tools

Test 1: List available tools

First, type `tools` and press Enter.

Input:

```plaintext
> tools
```

Expected Output:

```plaintext
  - echo: Echo back the input text.
  - write_file: Write content to a file.
```

This shows the two tools exposed by the server.

Test 2: Call the echo tool

First, type `call` and press Enter.

Input:

```plaintext
> call
```

The content of this lab is licensed under Apache 2.0

Input:

```plaintext
Tool name: echo
```

Then, enter the arguments as JSON and press Enter.

Input:

```plaintext
Arguments (as JSON, for example, {"text": "hello"}):
{"text": "Hello MCP!"}
```

Expected Output:

```plaintext
Result: Echo: Hello MCP!
```

Test 3: Create a file with write_file

First, type `call` and press Enter.

Input:

```plaintext
> call
```

Then, type `write_file` and press Enter.

Input:

The content of this lab is licensed under Apache 2.0

```plaintext
Tool name: write_file
```

Then, enter the arguments as JSON and press Enter.

Input:

```plaintext
Arguments (as JSON, for example, {"text": "hello"}):
{"path": "test.txt", "content": "Hello from MCP client!"}
```

Expected Output:

```plaintext
Result: Successfully wrote to test.txt
```

Verify the file was created. Click below to open it:

Open test.txt in IDE

You should see the content: `Hello from MCP client!`

The content of this lab is licensed under Apache 2.0

Test Resources

Test 4: List resource templates

First, type `resources` and press Enter.

Input:

```plaintext
> resources
```

Expected Output:

```plaintext
  • read_resource_file
    URI template: file://resources/{filename}
```

This shows the resource template pattern. You can substitute any filename for `{filename}`.

Test 5: Read project_info.txt

First, type `read` and press Enter.

Input:

```plaintext
> read
```

The content of this lab is licensed under Apache 2.0

Then, enter the URI and press Enter.

Input:

```plaintext
URI: file://resources/project_info.txt
```

Expected Output:

```plaintext
Project Name: MCP Client Lab
Version: 1.0.0
Description: A hands-on lab for building MCP clients with Python
Author: Skills Network
Status: Active
```

Test 6: Read README.md

First, type `read` and press Enter.

Input:

```plaintext
> read
```

Then, enter the URI and press Enter.

Input:

```plaintext
URI: file://resources/README.md
```

The content of this lab is licensed under Apache 2.0

Expected Output:

```plaintext
# MCP Client Lab

This lab teaches you how to build Model Context Protocol (MCP) clients

## What You'll Learn
- STDIO transport connections
- Tool invocation patterns
- Resource reading via URIs
- Prompt template usage

## Getting Started
Follow the lab instructions to build your first MCP client!
```

What's happening:

- The server exposes a resource template with the pattern: `file://resources/{filename}`
- You provide a concrete URI such as `file://resources/README.md`
- The server matches the URI to the template pattern
- The server extracts the parameter: `filename = "README.md"`
- The server calls `read_resource_file("README.md")` which reads and returns the file contents

The content of this lab is licensed under Apache 2.0

Test Prompts

Test 7: List prompt templates

First, type `prompts` and press Enter.

Input:

```plaintext
> prompts
```

Expected Output:

```plaintext
  • review_file: Generate a prompt to review a file's contents. (args:
```

This shows the available prompt template and its required argument.

Test 8: Get the review_file prompt

First, type `prompt` and press Enter.

Input:

```plaintext
> prompt
```

Then, type `review_file` and press Enter.

The content of this lab is licensed under Apache 2.0

Input:

```plaintext
Prompt name: review_file
```

Then, enter the arguments as JSON and press Enter.

Input:

```plaintext
Arguments (as JSON):
{"filename": "test.txt"}
```

Expected Output:

```plaintext
--- Prompt: Generate a prompt to review a file's contents. ---
user: Please review the file 'test.txt' and provide:

1. A summary of its contents
2. Key points or sections
3. Any suggestions for improvement
4. Overall quality assessment

Use the appropriate tools to read the file if needed.
```

What's happening:

- The client requests the `review_file` prompt template with `filename = "test.txt"` as the argument
- The server renders the template by substituting the filename parameter
- The server returns structured messages that form a complete prompt
- This rendered prompt could be sent to an LLM to generate a review
- The LLM could then use MCP tools (such as `read_file`) to access the file and

The content of this lab is licensed under Apache 2.0

When done testing, type `quit` and press Enter to exit the client.

The content of this lab is licensed under Apache 2.0

Understanding Resource Templates

Let's understand how resource templates work.

Static Resources vs Resource Templates

Static Resource:

```python
@mcp.resource("file://specific_file.txt")
def read_specific_file() -> str:
    return "Fixed content"
```

- Fixed URI: `file://specific_file.txt`
- No parameters
- Always returns same resource

Resource Template (what we used):

```python
@mcp.resource("file://resources/{filename}")
def read_resource_file(filename: str) -> str:
    # filename is extracted from the URI
    return Path(f"resources/{filename}").read_text()
```

- URI pattern: `file://resources/{filename}`
- Parameterized with `{filename}`
- Dynamic - returns different resources based on URI

The content of this lab is licensed under Apache 2.0

URI Parameter Extraction Flow

When you call `read_resource("file://resources/README.md")`:

1. **Client sends request:** Sends URI `file://resources/README.md` to the server
2. **Server matches pattern:** Identifies it matches the template `file://resources/{filename}`
3. **Server extracts parameter:** Extracts `filename = "README.md"` from the URI
4. **Server calls function:** Invokes `read_resource_file(filename="README.md")`
5. **Function executes:** Reads the file and returns the content
6. **Server responds:** Sends the file content back to the client

This pattern allows one resource template to dynamically expose many resources without defining each one individually!

The content of this lab is licensed under Apache 2.0

Understanding the Client Flow

Let's review how your minimal client works:

Connection Flow

plaintext

```
1. Launch client with server script path
   ↓
2. Create StdioServerParameters
   ↓
3. Launch server as subprocess via stdio_client
   ↓
4. Create ClientSession with read/write streams
   ↓
5. Call session.initialize() (MCP handshake)
   ↓
6. Ready for operations
```

The content of this lab is licensed under Apache 2.0

Example: Resource Reading Flow

plaintext

```
User: types "read"
   ↓
User: enters "file://resources/README.md"
   ↓
Client: calls session.read_resource(uri)
   ↓
[JSON-RPC request sent via stdin]
   ↓
Server: receives request with URI "file://resources/README.md"
   ↓
Server: matches URI to template pattern "file://resources/{filename}"
   ↓
Server: extracts parameter: filename = "README.md"
   ↓
Server: calls read_resource_file("README.md")
   ↓
Server: function reads file and returns content
   ↓
[JSON-RPC response sent via stdout]
   ↓
Client: receives result object
   ↓
Client: displays result.contents[0].text
```

Key Components

StdioServerParameters: Defines how to launch the server subprocess

stdio_client: Creates the STDIO transport, returns (read_stream, write_stream)

ClientSession: Manages MCP protocol - handles JSON-RPC, message IDs, protocol negotiation

AsyncExitStack: Ensures proper cleanup - closes streams and terminates subprocess

FastMCP: Simplifies server creation with decorators and automatic schema generation

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Testing Resource Templates with Additional Files

Let's see how resource templates work by adding another file to the resources directory.

Create a new file in the `resources/` directory.

Click below to open the file editor:

Open notes.txt in IDE

Add the following content:

plaintext

```plaintext
Lab Notes
=========
- MCP uses JSON-RPC 2.0
- STDIO transport is great for local development
- Resource templates use URI patterns with {parameters}
- Tools are actions, Resources are data
```

Save the file (`File` → `Save` or `Ctrl+S`).

Now test reading it with your client:

First, type `read` and press Enter.

Input:

plaintext

```plaintext
> read
```

The content of this lab is licensed under Apache 2.0

Input:

```plaintext
URI: file://resources/notes.txt
```

Expected Output:

```plaintext
Lab Notes
=========
- MCP uses JSON-RPC 2.0
- STDIO transport is great for local development
- Resource templates use URI patterns with {parameters}
- Tools are actions, Resources are data
```

What this demonstrates: The resource template `file://resources/{filename}` we defined in the server automatically exposes any file you add to the resources directory. You didn't need to modify the server code - the template pattern handles it dynamically!

The content of this lab is licensed under Apache 2.0

Extending the Client

Here are some simple extensions to try. Each extension shows which file to modify and provides full code.

Extension 1: Add a history log

Modify: `mcp_client.py`

Update the `__init__` method:

```python
def __init__(self):
    self.session = None
    self.exit_stack = AsyncExitStack()
    self.history = []  # Add this line
```

Update the `call_tool` method:

```python
async def call_tool(self, tool_name: str, arguments: dict):
    result = await self.session.call_tool(tool_name, arguments)
    self.history.append({"operation": "tool", "name": tool_name})  # A
    return result
```

Add a new command in the `run` method's command handling section:

The content of this lab is licensed under Apache 2.0

python

```python
                elif cmd == "history":
                    if self.history:
                        print("  Operation History:")
                        for i, entry in enumerate(self.history, 1):
                            print(f"    {i}. {entry['operation']}: {en
                    else:
                        print("  No operations yet")
```

Update the welcome message to include the new command:

python

```python
print("Commands: tools | call | resources | read | prompts | prompt | |
```

☐ Click below to see the modified `run` method with history support

▶ Full run method with history

Extension 2: Add better error messages

Modify: `mcp_client.py`

Update the error handling in the `run` method:

The content of this lab is licensed under Apache 2.0

python

```python
            except json.JSONDecodeError:
                print("  Error: Invalid JSON format")
                print("  Hint: Use double quotes, for example, {\"text
            except Exception as e:
                print(f"  Error: {e}")
                if "not found" in str(e).lower():
                    print("  Hint: Check the resource URI or filename"
```

☐ Click below to see the complete code for `mcp_client.py` with better error messages

▶ Full code

Extension 3: Add a help command

Modify: `mcp_client.py`

Add this to the command handling section in the `run` method:

python

```python
                elif cmd == "help":
                    print("""
Available Commands:
-------------------
tools       - List available tools
call        - Invoke a tool
resources   - List resource templates
read        - Read a resource by URI
prompts     - List prompt templates
prompt      - Get a rendered prompt
help        - Show this help message
quit        - Exit the client
                    """)
```

The content of this lab is licensed under Apache 2.0

Update the welcome message:

```python
print("Commands: tools | call | resources | read | prompts | prompt | |
```

☐ Click below to see the complete code for `mcp_client.py` with help command

▶ Full code

The content of this lab is licensed under Apache 2.0

Best Practices

Keep these practices in mind for MCP client development:

1. Always use AsyncExitStack

Ensures cleanup even when errors occur:

```python
self.exit_stack = AsyncExitStack()
# ... use it for all context managers
await self.exit_stack.aclose()  # Cleans everything
```

2. Initialize before operations

Always call `await session.initialize()` after creating the session. This performs the MCP handshake to negotiate protocol version and capabilities with the server.

3. Handle JSON parsing errors

When accepting JSON input from users:

```python
try:
    args = json.loads(user_input)
except json.JSONDecodeError:
    print("Invalid JSON format")
    return
```

The content of this lab is licensed under Apache 2.0

4. Understand resource templates

- Resource templates use URI patterns with `{parameters}` such as `file://resources/{filename}`
- Discover available templates using `list_resource_templates()`
- Create concrete URIs by substituting actual values for parameters
- One template can dynamically expose many resources without individual definitions

5. Use tool schemas

Check tool schemas from `list_tools()` to know:

- Required vs optional arguments
- Argument types and descriptions
- What the tool does

6. Validate URIs

When reading resources, verify the URI matches an available template pattern to avoid errors.

7. Close cleanly

Always cleanup in a `finally` block to properly close connections and release resources, even when errors occur.

The content of this lab is licensed under Apache 2.0

Conclusion

Congratulations! You've built a minimal but fully functional MCP client.

What you've accomplished

✅ Created a lightweight MCP client with STDIO transport ✅ Connected to a FastMCP server ✅ Discovered and invoked tools (echo, write_file) ✅ Listed and read resources via URI templates ✅ Retrieved and used prompt templates ✅ Built a simple command-line interface ✅ Understood resource template patterns ✅ Worked with real resource files

Key concepts

STDIO Transport - Local communication via stdin/stdout, perfect for development

ClientSession - Manages all MCP protocol details (JSON-RPC, message IDs, and so on)

FastMCP - Simplifies server creation with decorators and automatic schema generation

Tools - Server actions the client can invoke with arguments

Resource Templates - URI patterns such as `file://resources/{filename}` that dynamically expose multiple resources

Prompts - Server templates the client can render with arguments

You now have a solid foundation for building MCP-enabled applications!

Author(s)

Wojciech \"Victor\" Fulmyk

The content of this lab is licensed under Apache 2.0

© IBM Corporation. All rights reserved.

Lab Setup

Let's set up your development environment.

Create Virtual Environment

bash

```bash
python3.11 -m venv mcp_client_env
source mcp_client_env/bin/activate
```

Run

Install Dependencies

Install the MCP SDK and FastMCP for simplified server creation:

bash

```bash
pip install mcp==1.16.0 fastmcp==2.12.5
```

Run

Create Project Structure

bash

```bash
mkdir mcp_client_lab
cd mcp_client_lab
mkdir resources
```

The content of this lab is licensed under Apache 2.0

We'll create the server, client, and resource files in the next steps.

← Build a Custom MCP Clie…

Create Resource Files →

The content of this lab is licensed under Apache 2.0